

# Amazon Lens — System Architecture

---

**Amazon Lens Team**

June 15, 2026

# CONTENTS

|                                               |       |
|-----------------------------------------------|-------|
| Amazon Lens — System Architecture             | ..... |
| 1. Overview                                   | ..... |
| 2. Tech Stack                                 | ..... |
| 3. Repository Structure                       | ..... |
| 4. Frontend Architecture                      | ..... |
| 4.1 State Management                          | ..... |
| 4.2 Pages (19 Routes)                         | ..... |
| 4.3 Key Components                            | ..... |
| 4.4 Proxy Configuration (Vite)                | ..... |
| 5. Backend Architecture                       | ..... |
| 5.1 API Modules (10 Route Files, prefix /api) | ..... |
| 5.2 Real-Time Events (Socket.IO)              | ..... |
| 5.3 Middleware                                | ..... |
| 6. Database Architecture                      | ..... |
| 6.1 Collections                               | ..... |
| 6.2 File-Based Storage                        | ..... |
| 6.3 Client-Side Only (localStorage)           | ..... |
| 7. External Integrations                      | ..... |
| 8. Core Business Logic                        | ..... |
| 8.1 TrustLens™ Score Formula                  | ..... |
| 8.2 Eco Score Formula                         | ..... |

|                                        |       |
|----------------------------------------|-------|
| 8.3 Fake Discount Detection            | ..... |
| 8.4 Bundle Complement Rules            | ..... |
| 9. Key Data Flows                      | ..... |
| 9.1 Product Page Load                  | ..... |
| 9.2 Smart Search (NL Query)            | ..... |
| 9.3 AI Bundle Generation               | ..... |
| 9.4 Amazon Sense Reorder Flow          | ..... |
| 9.5 Co-Planner Real-Time Collaboration | ..... |
| 9.6 Returns Flow                       | ..... |
| 10. Authentication                     | ..... |
| 11. Infrastructure & Ports             | ..... |
| 12. Feature Summary                    | ..... |

# Amazon Lens — System Architecture

**Document Type:** Product Requirements Document — System Architecture

**Project:** Amazon Lens

**Version:** 1.0

**Date:** June 2026

## 1. Overview

Amazon Lens is a full-stack e-commerce intelligence platform that layers trust scoring, behavioral personalization, AI-driven recommendations, and real-time owner interaction on top of a simulated Amazon-like storefront. The system is built as a **monorepo** with a React SPA frontend and an Express.js API backend communicating via REST and WebSockets.

## 2. Tech Stack

| Layer         | Technology                                                |
|---------------|-----------------------------------------------------------|
| Frontend      | React 18, Vite 5, Tailwind CSS 3.3, React Router v6       |
| Backend       | Node.js, Express.js 4.18                                  |
| Database      | MongoDB 7 (Mongoose ODM)                                  |
| Real-Time     | Socket.IO 4.8                                             |
| AI / LLM      | Groq API — llama-3.3-70b-versatile , llama-3.1-8b-instant |
| Auth          | JWT (bcryptjs, 7-day expiry, stored in localStorage)      |
| External APIs | Groq, Open Library API, DummyJSON (static)                |
| Charts        | Recharts 2.10                                             |
| Build Tooling | Vite 5 (client), nodemon (server), concurrently (root)    |

## 3. Repository Structure

```
amazon-lens/
├── client/                                # React 18 + Vite SPA (port 5173)
│   └── src/
│       ├── pages/                        # 19 route pages
│       ├── components/                  # Reusable UI components
│       ├── contexts/                    # 11 Context API state stores
│       └── utils/                        # format helpers, socket client
├── server/                               # Express.js backend (port 5001)
│   ├── routes/                          # 10 API route modules
│   ├── models/                          # Mongoose schemas (User, DnaProfile)
│   ├── middleware/                      # JWT auth middleware
│   ├── utils/                           # Groq client, trust formula
│   └── data/                            # Static mock data (products, companies)
├── scripts/                             # Utility / seeding scripts
├── docs/                                # Documentation
└── package.json                          # Root monorepo (concurrently)
```

**Start command:** `npm run dev` launches both client (5173) and server (5001) in parallel.

---

## 4. Frontend Architecture

### 4.1 STATE MANAGEMENT

All shared state is managed via 11 React Context APIs — no Redux or external state library.

| Context               | Storage            | Purpose                       |
|-----------------------|--------------------|-------------------------------|
| AuthContext           | localStorage       | JWT session, user identity    |
| CartContext           | localStorage       | Cart items (not synced to DB) |
| OrdersContext         | localStorage       | Order history                 |
| WishlistContext       | localStorage       | Saved items                   |
| SenseContext          | API + localStorage | DNA reorder predictions       |
| DnaContext            | API                | Behavioral event tracking     |
| WitnessContext        | Socket.IO          | Live owner chat state         |
| CoPlannerContext      | Socket.IO          | Shared list real-time updates |
| SustainabilityContext | memory             | Eco mode toggle, rankings     |
| ReviewsContext        | API                | User review submissions       |
| HistoryContext        | localStorage       | Browsing history              |

## 4.2 PAGES (19 ROUTES)

| Route              | Purpose                                               |
|--------------------|-------------------------------------------------------|
| /                  | Homepage — deals, Sense popup, bundle recommendations |
| /s                 | Search results with bundle detection                  |
| /dp/:productId     | Product page — TrustLens, WitnessPanel, price history |
| /cart              | Cart with Sense Soon reorder tab                      |
| /checkout          | Order placement (stub)                                |
| /orders            | Order history + returns initiation                    |
| /wishlist          | Saved items                                           |
| /bundles           | Curated and AI-generated bundles                      |
| /bundles/:bundleId | Bundle detail with add-to-cart                        |
| /co-planner        | Shared shopping list (real-time via Socket.IO)        |
| /sustainability    | Eco profile and company eco rankings                  |
| /smart-search      | Natural language search with Groq AI                  |
| /account           | User profile and settings                             |
| /witness           | Owner chat portal (live witness mode)                 |
| /my-reviews        | User-submitted reviews                                |
| /returns           | AI-powered return swap suggestions                    |
| /history           | Browsing history                                      |
| /login, /signup    | Authentication pages                                  |

## 4.3 KEY COMPONENTS

- **ProductCard** — Displays TrustLens badge, price, eco label, Sense match score
- **TrustCard** — Expandable 5-dimension trust breakdown modal

- **WitnessPanel** — Live owner list with real-time chat buttons
- **SenseMatchPanel** — Personalized match score (80+ = recommended)
- **SensePopup** — Reorder prediction popup (3s delay on homepage)
- **BundleCard** — Bundle with AI-generated confidence score and reason
- **AmazonLensAssistant** — AI chat sidebar (Groq-powered)

## 4.4 PROXY CONFIGURATION (VITE)

```
/api/*      → http://localhost:5001  (REST)
/socket.io/* → http://localhost:5001  (WebSocket upgrade)
```

---



## 5. Backend Architecture

### 5.1 API MODULES (10 ROUTE FILES, PREFIX `/API`)

| Module                      | Key Endpoints                                                                                    | Purpose                                                    |
|-----------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <code>auth.js</code>        | <code>POST /signup</code> , <code>POST /login</code>                                             | User registration + JWT issuance                           |
| <code>products.js</code>    | <code>GET /products</code> , <code>GET /products/:id</code> , <code>POST /products/search</code> | Product catalog, search, bundle detection                  |
| <code>sense.js</code>       | 10 endpoints                                                                                     | DNA profiling, reorder predictions, risk scoring, match-AI |
| <code>witness.js</code>     | <code>GET /witness/:id</code> , <code>POST /witness/chat</code>                                  | Owner personas + Groq AI chat fallback                     |
| <code>bundles.js</code>     | <code>GET /bundles</code> , <code>POST /bundles/ai</code>                                        | Curated + Groq AI-generated bundles                        |
| <code>companies.js</code>   | 6 endpoints                                                                                      | Seller profiles, eco scoring, refresh metrics              |
| <code>coPlanner.js</code>   | 8 endpoints                                                                                      | Shared plans, roles, expense splitting, invites            |
| <code>smartSearch.js</code> | <code>POST /smart-search</code>                                                                  | NL query + Groq intent classification                      |
| <code>returns.js</code>     | <code>POST /returns/suggestions</code>                                                           | AI-powered return swap suggestions                         |
| <code>dna.js</code>         | <code>POST /dna/event</code> , <code>GET /dna/profile</code>                                     | DNA profile CRUD                                           |

**Health check:** `GET /api/health` → `{ status: "ok" }`

## 5.2 REAL-TIME EVENTS (SOCKET.IO)

### Witness channel:

| Event                         | Direction       | Purpose                                  |
|-------------------------------|-----------------|------------------------------------------|
| witness:online                | client → server | Register owner as live for a product     |
| witness:offline               | client → server | Unregister owner                         |
| witnesses:subscribe           | client → server | Buyer watches witness list for a product |
| witnesses:updated             | server → client | Broadcast when witnesses come/go         |
| chat:request                  | client → server | Buyer initiates chat                     |
| chat:accept /<br>chat:decline | client → server | Witness responds                         |
| chat:message                  | bidirectional   | In-session message                       |
| chat:end                      | bidirectional   | Close chat                               |
| chat:timeout                  | server → client | Auto-timeout after 30 seconds            |

### Co-Planner channel:

| Event        | Direction       | Purpose                               |
|--------------|-----------------|---------------------------------------|
| coplan:join  | client → server | Subscribe to plan room                |
| coplan:leave | client → server | Unsubscribe from plan room            |
| plan:updated | server → client | Broadcast item changes to all members |

## 5.3 MIDDLEWARE

- **authMiddleware.js** — Soft JWT validation; invalid/missing tokens proceed as guest
  - **cors()** — All origins allowed (local dev)
  - **express.json()** — JSON body parsing
- 

## 6. Database Architecture

**Engine:** MongoDB (local — `mongodb://127.0.0.1:27017/amazon-lens`)

### 6.1 COLLECTIONS

**users**

| Field      | Type   | Notes                    |
|------------|--------|--------------------------|
| name       | String | Required                 |
| email      | String | Unique, lowercase        |
| password   | String | bcrypt-hashed, 10 rounds |
| timestamps | Date   | createdAt, updatedAt     |

**dnaprofiles**

| Field                  | Type     | Notes                            |
|------------------------|----------|----------------------------------|
| userId                 | ObjectId | Ref → User (logged-in)           |
| guestId                | String   | UUID (anonymous)                 |
| events                 | Array    | Capped at 200 entries            |
| preferredBrands        | Array    | <code>[{brand, score}]</code>    |
| preferredCategories    | Array    | <code>[{category, score}]</code> |
| budgetRange            | Object   | <code>{min, max, avg}</code>     |
| sustainabilityAffinity | Number   | 0.0–1.0                          |
| returnPatterns         | Array    | Per-category return rates        |
| returnedProductIds     | Array    | Product IDs of returned items    |
| purchasedProductIds    | Array    | Product IDs of purchases         |

## 6.2 FILE-BASED STORAGE

| File                                | Contents                              |
|-------------------------------------|---------------------------------------|
| <code>coplanner-plans.json</code>   | All co-planner plan data (persistent) |
| <code>coplanner-invites.json</code> | Invite tokens, expiry (7-day default) |

## 6.3 CLIENT-SIDE ONLY (LOCALSTORAGE)

Cart, orders, wishlist, and browsing history are stored in the browser and are not synced to MongoDB.

## 7. External Integrations

| Service          | Model / Endpoint                     | Used For                                                        |
|------------------|--------------------------------------|-----------------------------------------------------------------|
| Groq API         | llama-3.3-70b-versatile              | Bundle generation, review analysis, fake review detection       |
| Groq API         | llama-3.1-8b-instant                 | Smart search intent, witness chat, return suggestions, match-AI |
| Open Library API | openlibrary.org/subjects/{slug}.json | Live book catalog (fetched once at server startup)              |
| DummyJSON        | Static import ( djProducts.js )      | Expanded product catalog beyond Amazon Lens native products     |

**Groq fallback:** All Groq-dependent routes return hardcoded responses if the API is unavailable or rate-limited. Rate limit: llama-3.1-8b-instant — 500k tokens/day.

## 8. Core Business Logic

### 8.1 TRUSTLENS™ SCORE FORMULA

$$\text{Score} = \text{round}((0.50 \times \text{Rs} + 0.30 \times \text{Kp} + 0.20 \times \text{Ri}) \times 100)$$

|    |   |               |   |                     |       |
|----|---|---------------|---|---------------------|-------|
| Rs | = | Review Score  | = | (avgRating - 1) / 4 | [0–1] |
| Kp | = | Keep Rate     | = | 1 - returnRate      | [0–1] |
| Ri | = | Reorder Index | = | sellerRating / 4    | [0–1] |

User return penalty: -2 pts per return (max -30 pts)

Final score:  $\text{max}(5, \text{min}(98, \text{score} - \text{penalty}))$

Status thresholds:

|           |   |          |               |
|-----------|---|----------|---------------|
| $\geq 75$ | → | VERIFIED | (green badge) |
| 50–74     | → | TRUSTED  | (blue badge)  |
| $< 50$    | → | no badge |               |

### 8.2 ECO SCORE FORMULA

$$\begin{aligned} \text{baseScore} &= \text{carbon}(25\%) + \text{renewable}(20\%) + \text{recyclable}(20\%) \\ &\quad + \text{supply\_chain}(20\%) + \text{longevity}(15\%) \\ \text{ecoScore} &= \text{round}(\text{min}(99, \text{baseScore} + \text{min}(10, \text{certCount} \times 2.5))) \end{aligned}$$

Labels:

|           |   |                |               |
|-----------|---|----------------|---------------|
| $\geq 90$ | → | Climate Leader | (dark green)  |
| $\geq 80$ | → | Eco Advanced   | (green)       |
| $\geq 70$ | → | Eco Conscious  | (teal)        |
| $\geq 60$ | → | Eco Aware      | (light green) |
| $\geq 50$ | → | Progressing    | (yellow)      |
| $\geq 40$ | → | Developing     | (orange)      |
| $< 40$    | → | Early Stage    | (gray)        |

### 8.3 FAKE DISCOUNT DETECTION

A product is flagged as a fake discount if the listed MRP exceeds the actual sale price in 8 or more of the past 12 months of price history.

## 8.4 BUNDLE COMPLEMENT RULES

| Trigger Category | Suggested Complements                               |
|------------------|-----------------------------------------------------|
| Coffee           | Kitchen appliances (kettle, induction, cooker)      |
| TV               | Audio (soundbar) + Streaming (Fire Stick)           |
| Phone            | Audio (earbuds) + Accessories (power bank, USB hub) |
| Gaming           | Keyboard, Mouse, Monitor, Desk Mat                  |
| Office           | Lamp, Webcam, Cable management, USB hub             |

## 9. Key Data Flows

### 9.1 PRODUCT PAGE LOAD

```

Client visits /dp/p001
├── GET /api/products/p001           product metadata
├── POST /api/sense/seller-trust    TrustLens score breakdown
├── GET /api/witness/p001          owner personas
├── GET /api/sense/risk/p001       purchase risk flags
├── POST /api/sense/match-ai       personalized match score
(logged-in users)
├── socket.emit("witnesses:subscribe") live owner availability
(Socket.IO)

```

## 9.2 SMART SEARCH (NL QUERY)

```
Client: POST /api/smart-search { query: "home theatre setup under 40000"
}
```

↓

```
smartSearch.js:
```

1. Call Groq llama-3.1-8b to classify intent → { type, keywords, priceMax, category }
2. If type = "bundle": detect bundle keyword → return bundle + component products
3. Else: score all products by name / brand / category match, return top results

```
Response: { type: "product"|"bundle", products: [...], bundleProducts?: [...] }
```

## 9.3 AI BUNDLE GENERATION

```
Client: POST /api/bundles/ai { recentOrder, olderOrders, allPurchasedIds, history }
```

↓

```
bundles.js:
```

1. Build prompt with recent order + full catalog + category complement rules
2. Call Groq llama-3.1-8b (max\_tokens: 900, temp: 0.2)
3. Parse response: shoppingContext, bundles[.]{id, title, reason, goal, items, confidence}
4. Filter: valid product IDs only, exclude already-owned items
5. Return 2 bundles (3-5 items each)



## 9.4 AMAZON SENSE REORDER FLOW

1. Events tracked client-side:  
 POST /api/sense/event { type: "purchase"|"view"|"cart\_add"|"return",  
 productId, ... }
2. Backend (sense.js):
  - Get or create DnaProfile for userId / guestId
  - Append event (capped at 200)
  - Recompute: preferredBrands, budgetRange, returnPatterns
3. Prediction served:  
 GET /api/sense/predictions → senseItems: [{ name, daysOverdue, ... }]
  - Shown as popup on homepage after 3s
  - Shown as "Sense Soon" tab in cart (items where daysOverdue ≤ 0)

## 9.5 CO-PLANNER REAL-TIME COLLABORATION

Create plan: POST /api/co-planner/create { title, members }  
 → planId, inviteToken, shareLink

Invite: POST /api/co-planner/:planId/invite  
 → token (embedded in QR code URL, expires in 7 days)

Join via QR: PUT /api/co-planner/:planId/accept-invite  
 → adds user to plan.members with role "member"

Live sync:  
 Client → socket.emit("coplan:join", { planId })  
 Server → socket.join(`coplan:\${planId}`)  
 On change: io.to(`coplan:\${planId}`).emit("plan:updated", {...})  
 All members receive the update instantly

## 9.6 RETURNS FLOW

1. User selects product to return on /returns

2. GET /api/sense/risk/:productId show risk breakdown

3. POST /api/returns/suggestions { productId, productName, brand, category }
- Backend (returns.js):
  - Build alternatives catalog (exclude returned product + same brand)
  - Prompt Groq: "Pick 3 best swap alternatives"
  - Score: same category > price match > rating > different brand
  - Return 3 suggestions with swap reasons

# 10. Authentication

Mechanism: Stateless JWT

| Step          | Detail                                                                       |
|---------------|------------------------------------------------------------------------------|
| Signup        | POST /api/auth/signup — bcrypt hash (10 rounds), issue JWT                   |
| Login         | POST /api/auth/login — bcrypt.compare, issue JWT                             |
| Token storage | localStorage key "token"                                                     |
| Token expiry  | 7 days                                                                       |
| Header format | Authorization: Bearer <token>                                                |
| Guest support | If token is absent/invalid, requests proceed with guestId (UUID query param) |

Auth is **soft** — all routes function for guests. Co-Planner routes require a valid JWT for write operations.

## 11. Infrastructure & Ports

| Service        | Port  | URL                                                |
|----------------|-------|----------------------------------------------------|
| React Client   | 5173  | <code>http://localhost:5173</code>                 |
| Express Server | 5001  | <code>http://localhost:5001</code>                 |
| MongoDB        | 27017 | <code>mongodb://127.0.0.1:27017/amazon-lens</code> |

Environment variables (server `.env`):

| Variable                  | Purpose                            |
|---------------------------|------------------------------------|
| <code>MONGODB_URI</code>  | MongoDB connection string          |
| <code>JWT_SECRET</code>   | JWT signing secret                 |
| <code>GROQ_API_KEY</code> | Groq LLM API key                   |
| <code>CLIENT_URL</code>   | CORS origin whitelist              |
| <code>PORT</code>         | Express server port (default 5001) |

## 12. Feature Summary

| Feature       | Core Tech                                                | Key Files                                                          |
|---------------|----------------------------------------------------------|--------------------------------------------------------------------|
| TrustLens™    | Trust formula + customer DB + Groq review analysis       | <code>trustFormula.js</code> , <code>sense.js</code>               |
| WitnessPanel™ | Socket.IO real-time chat + Groq AI fallback              | <code>witness.js</code> ,<br><code>WitnessContext.jsx</code>       |
| Amazon Sense™ | DNA profiling + event tracking + reorder predictions     | <code>DnaProfile</code> model, <code>sense.js</code>               |
| Green Choice™ | Eco scoring + company ratings + sustainability dashboard | <code>companies.js</code> ,<br><code>SustainabilityPage.jsx</code> |
| Smart Bundles | Groq AI generation + category complement rules           | <code>bundles.js</code> ,<br><code>BundleDetailPage.jsx</code>     |
| Co-Planner    | Shared lists + roles + real-time Socket.IO + QR invite   | <code>coPlanner.js</code> ,<br><code>CoPlannerPage.jsx</code>      |
| Smart Search  | NL query + Groq intent classification + bundle detection | <code>smartSearch.js</code> ,<br><code>SearchResults.jsx</code>    |
| Returns AI    | Groq-powered swap suggestions                            | <code>returns.js</code> , <code>ReturnsPage.jsx</code>             |